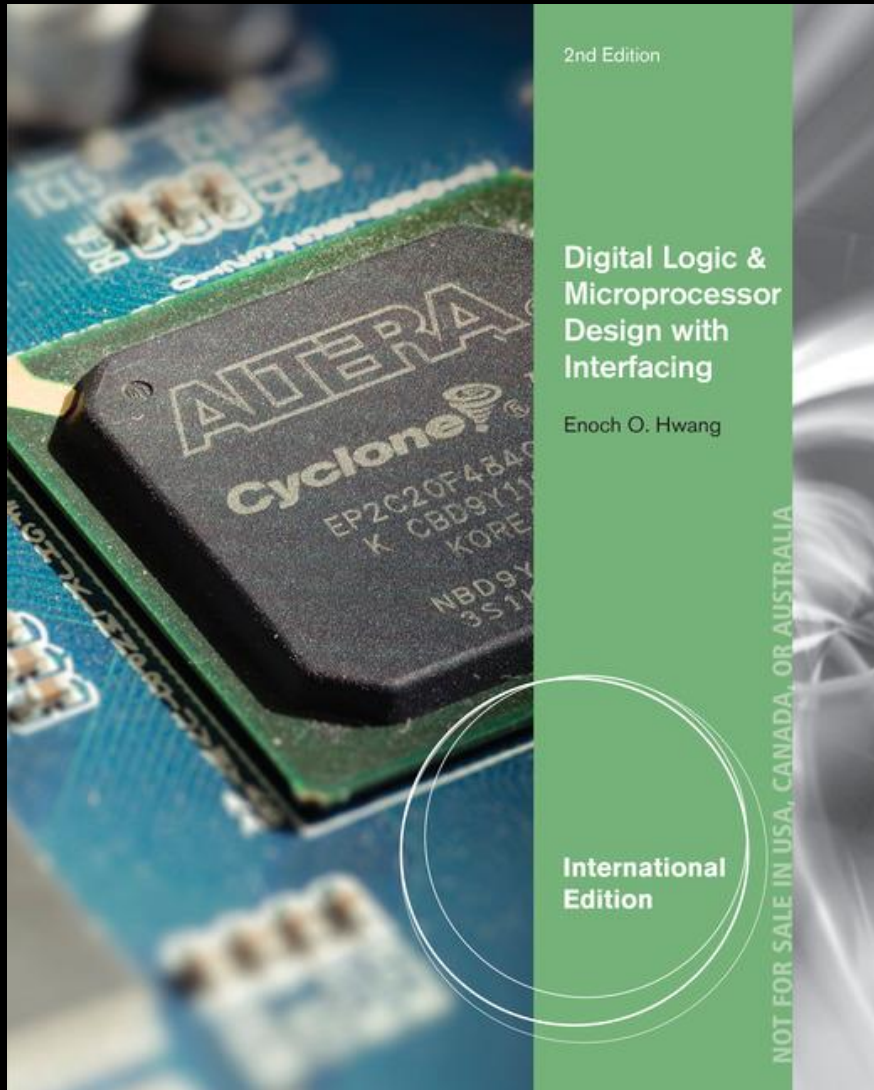




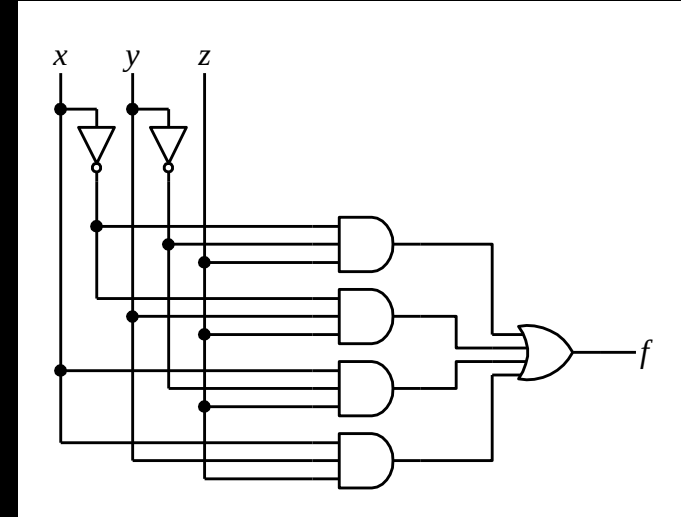
Chapter 3 Combinational Circuits

In this Chapter

- Analysis of Combinational Circuits
- Synthesis of Combinational Circuits
- Minimization
- Timing Hazards and Glitches
- Example: BCD to 7-segment Decoder
- Verilog Code for example

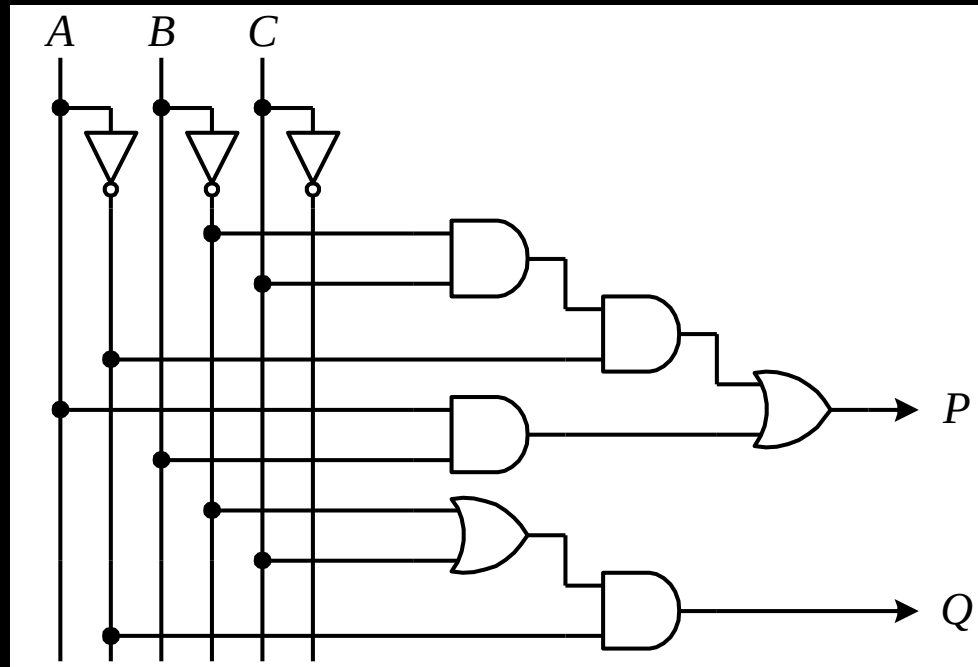
Analysis of Combinational Circuit

- Analysis
 - Given a circuit, we want to know its operation
- Describe operation using either
 - Truth table
 - Boolean function
- Steps to create truth table



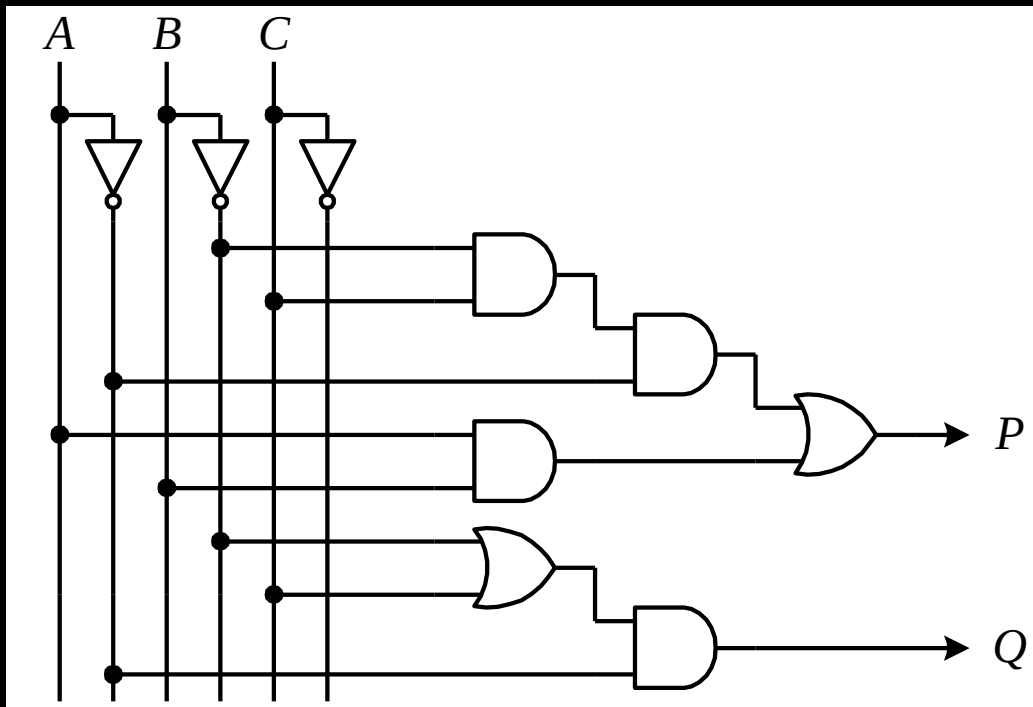
Analysis of Combinational Circuit

- Derive the truth table for this circuit:



Analysis of Combinational Circuit

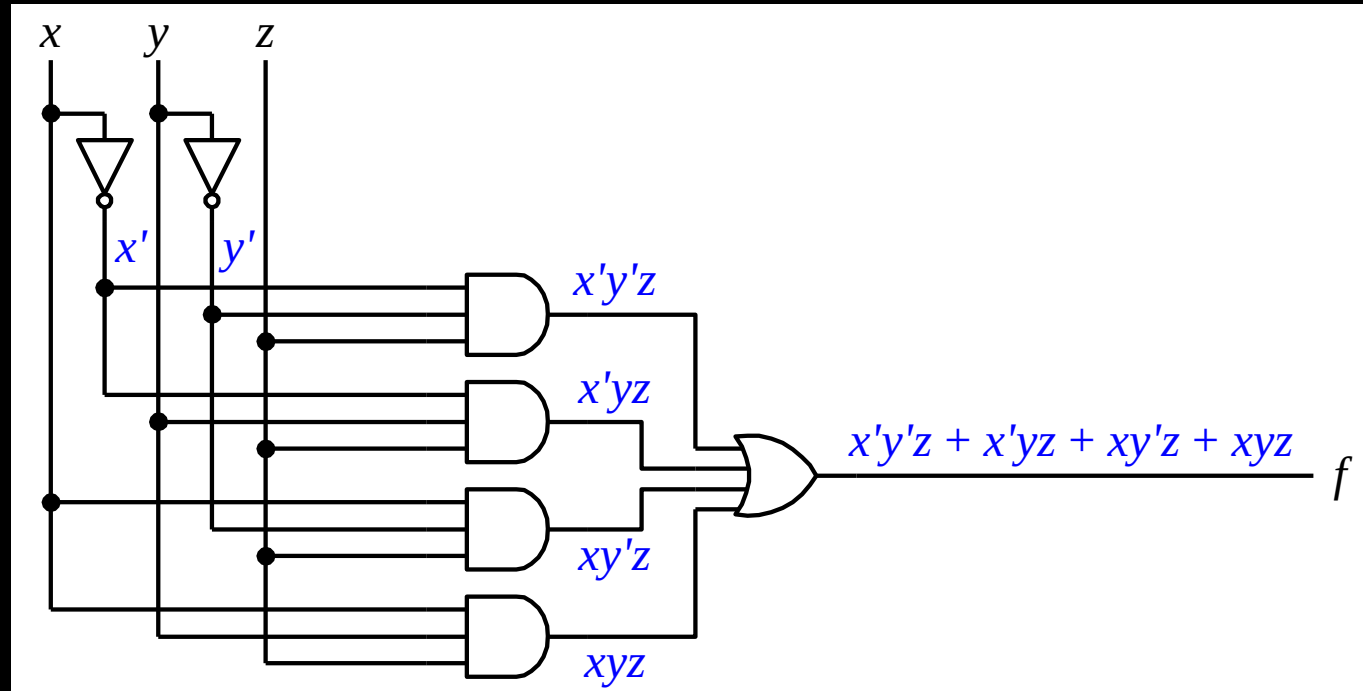
- Derive the truth table for this circuit:



<i>A</i>	<i>B</i>	<i>C</i>	<i>P</i>	<i>Q</i>
0	0	0	0	1
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	0
1	1	1	1	0

Analysis of Combinational Circuit

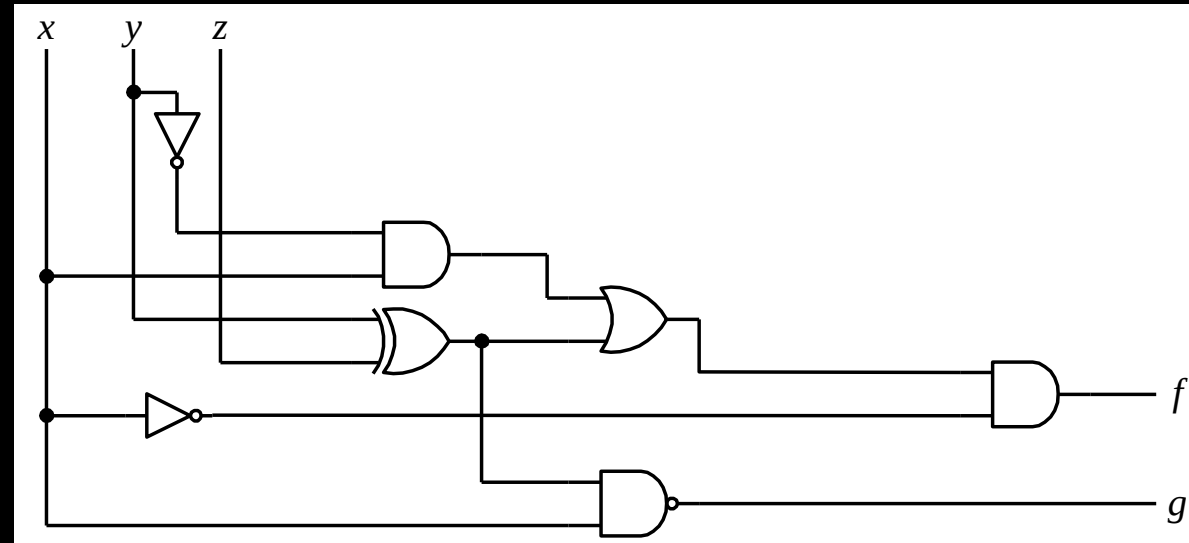
- Derive the Boolean function for this circuit:



$$f = x'y'z + x'yz + xy'z + xyz$$

Analysis of Combinational Circuit

- Derive the Boolean function for this circuit:

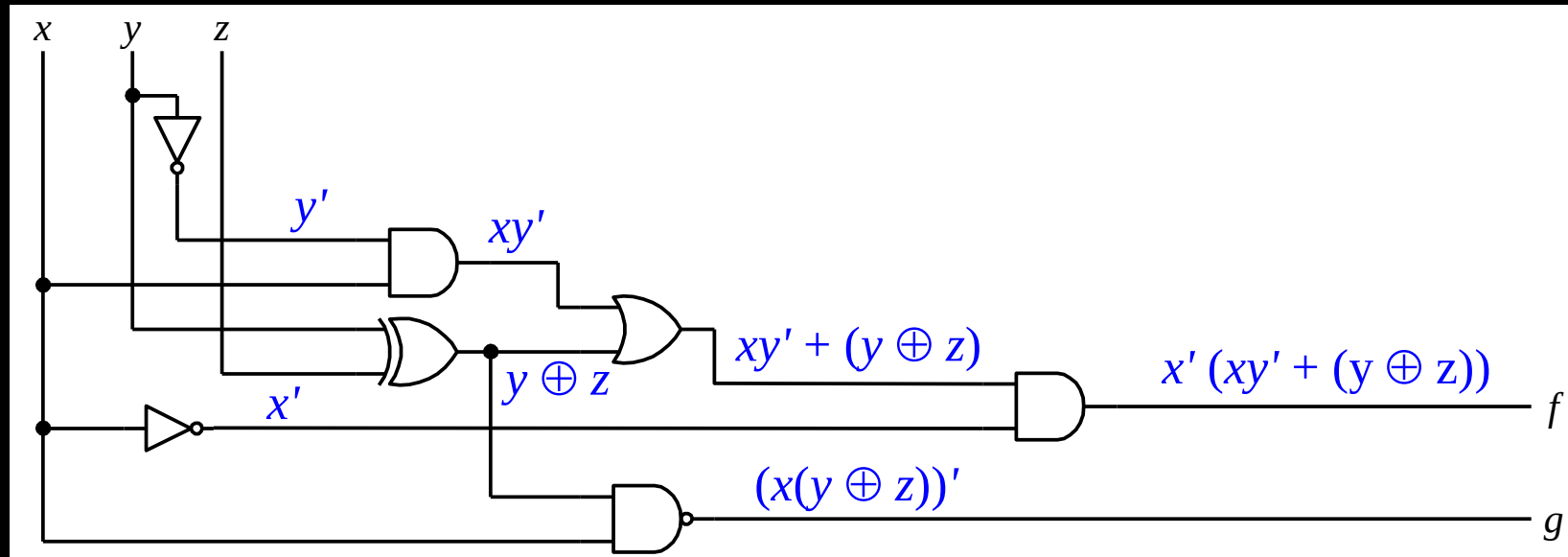


$$f = ?$$

$$g = ?$$

Analysis of Combinational Circuit

- Using Boolean function



$$f = x' (xy' + (y \oplus z))$$

$$g = (x(y \oplus z))'$$

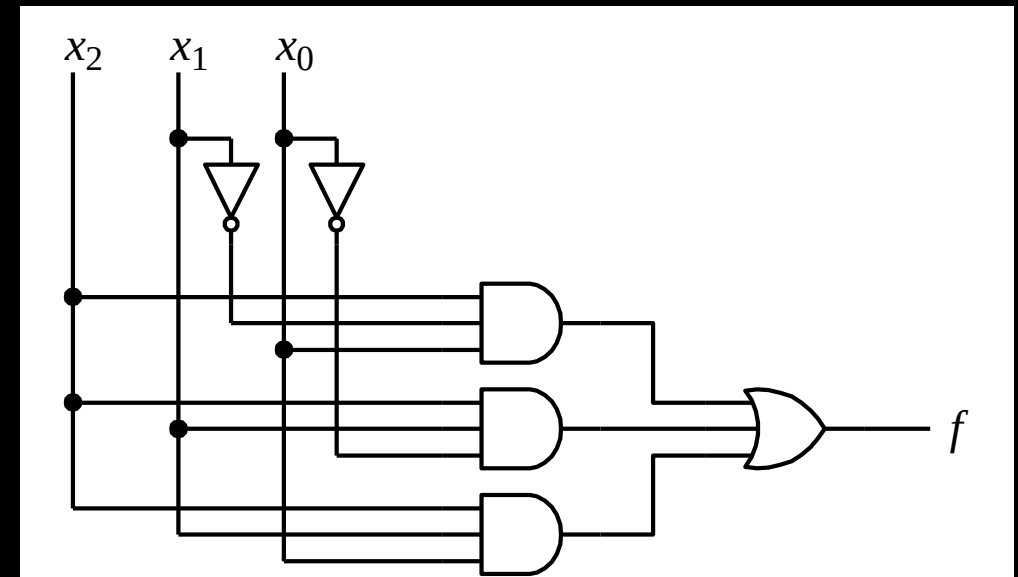
Synthesis of Combinational Circuit

- Synthesis is the reverse of Analysis
- Given the operation of a circuit with either a truth table or a Boolean equation
- Derive the circuit for it
- Example: construct a 3-bit unsigned comparator circuit for the “greater than or equal to 5” test

Synthesis of Combinational Circuit

- 3-bit “greater than or equal to 5” comparator circuit

Decimal number	Binary number			Output
	x_2	x_1	x_0	f
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1



$$f = x_2x_1x_0 + x_2x_1x_0' + x_2x_1'x_0$$

Synthesis of Combinational Circuit

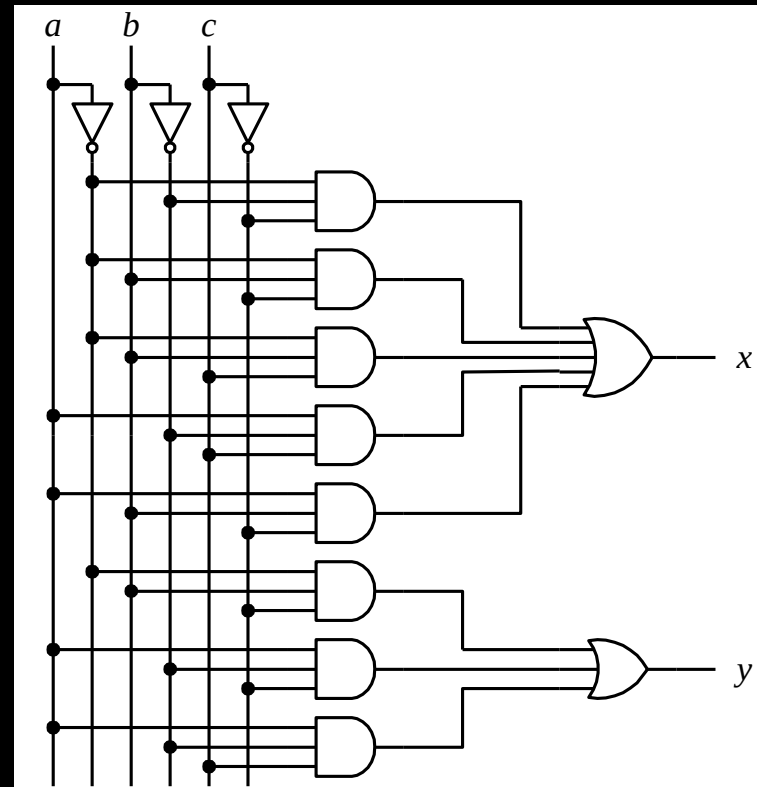
- Synthesize the circuit for this truth table

<i>a</i>	<i>b</i>	<i>c</i>	<i>x</i>	<i>y</i>
0	0	0	1	0
0	0	1	0	0
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	1
1	1	0	1	0
1	1	1	0	0

Synthesis of Combinational Circuit

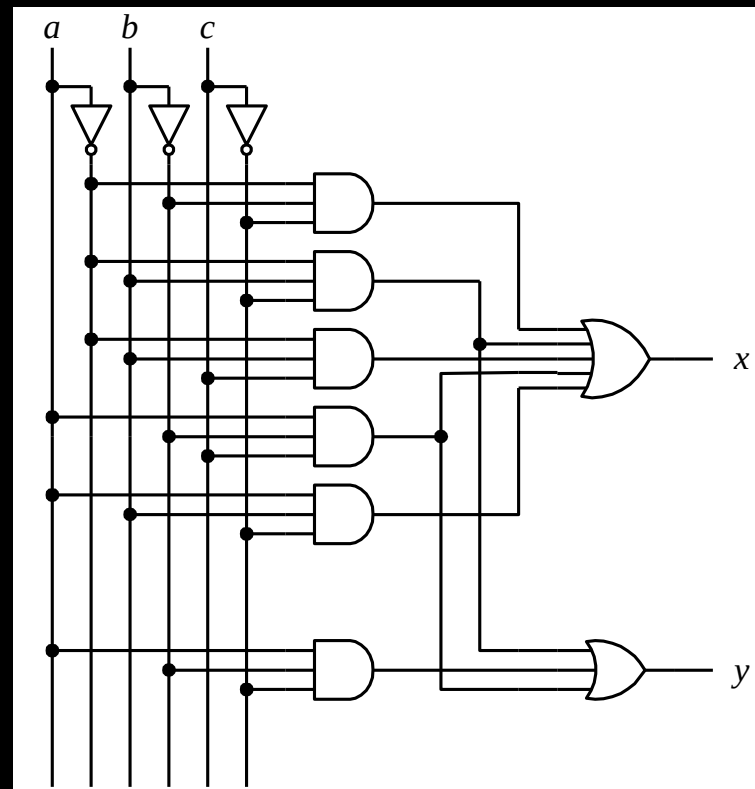
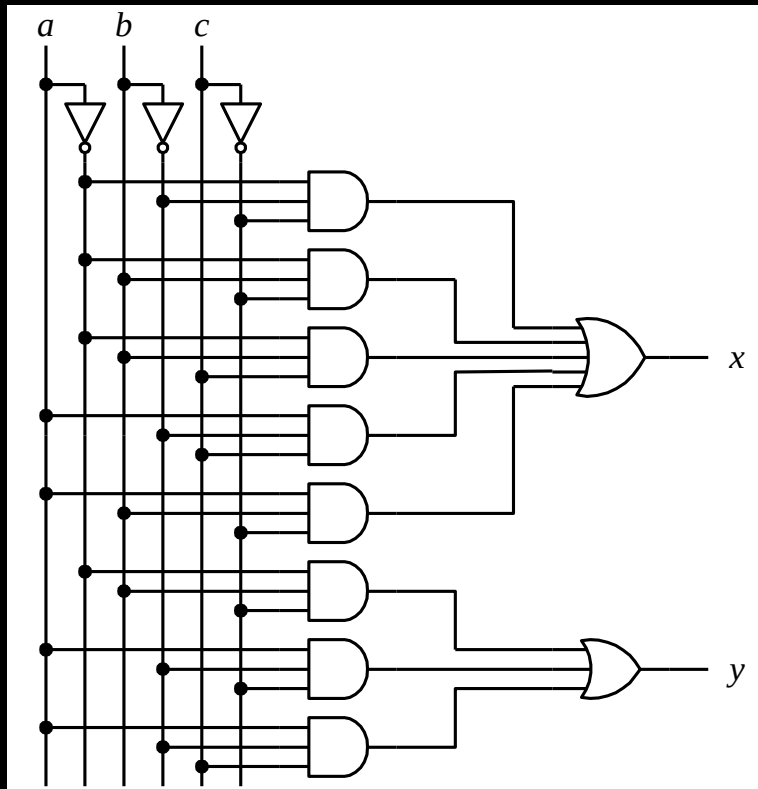
- Synthesize the circuit for this truth table

a	b	c	x	y
0	0	0	1	0
0	0	1	0	0
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	1
1	1	0	1	0
1	1	1	0	0



Synthesis of Combinational Circuit

- Optimize this circuit



Minimization of Combinational Circuits

- Optimize circuit size, speed, and power
- How to optimize? One to one map from Boolean to Circuit
- From Canonical to Standard to Non-Standard
- Methods
 - Boolean algebra – theoretical – difficult
 - Karnaugh maps – easy, but only for few variables
 - Tabulation method – best for computer

Boolean Algebra

- Minimize a sum-of-minterms format, reduce the number of product terms
- This is done by **factoring out the common variable(s)**.

$$\begin{aligned} F &= xy'z' + xyz' \\ &= xz'(y' + y) \\ &= xz' 1 \\ &= xz' \end{aligned}$$

Boolean Algebra

- Reducing the number of product terms
 - will reduce the number of inputs to the OR gate
 - will reduce the number of AND gates
- Reducing the number of variables
 - will reduce the number of inputs to the AND gate

Boolean Algebra

- Sometimes it is advantageous to duplicate a product term one or more times in an equation

$$\begin{aligned} F &= x'yz' + x'yz + xyz' \\ &= x'yz' + x'yz + xyz' + x'yz' \quad (\text{duplicate } x'yz') \\ &= (x'yz' + x'yz) + (x'yz' + xyz') \\ &= x'y(z' + z) + yz'(x' + x) \\ &= x'y \cdot 1 + yz' \cdot 1 \\ &= x'y + yz' \\ &= y(x' + z') \end{aligned}$$

Karnaugh Maps (K-Maps)

- A **K-map** provides a simple and straightforward procedure for combining product terms
- Minterms are grouped so that we can easily see which of the minterms can be combined
- Adjacent squares differ in the value of only one bit
- Use minterm numbers for easy reference

Karnaugh Maps (K-Maps)

		y	
		0	1
x	0	$x'y'$	$x'y$
	1	xy'	xy

		yz			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'

		yz			
		00	01	11	10
wx	00	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
	01	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$
	11	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$
	10	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$

		$v = 0$				$v = 1$			
		yz				yz			
wx	00	$v'w'x'y'z'$	$v'w'x'y'z$	$v'w'x'yz$	$v'w'x'yz'$	$vw'x'y'z'$	$vw'x'y'z$	$vw'x'yz$	$vw'x'yz'$
	01	$v'w'xy'z'$	$v'w'xy'z$	$v'w'xyz$	$v'w'xyz'$	$vw'xy'z'$	$vw'xy'z$	$vw'xyz$	$vw'xyz'$
	11	$v'wxy'z'$	$v'wxy'z$	$v'wxyz$	$v'wxyz'$	$vwxy'z'$	$vwxy'z$	$vwxyz$	$vwxyz'$
	10	$v'wx'y'z'$	$v'wx'y'z$	$v'wx'yz$	$v'wx'yz'$	$vwx'y'z'$	$vwx'y'z$	$vwx'yz$	$vwx'yz'$

Equality of Boolean equations and K-map

$$F = x'y' + x'y + xy$$

		y	
		0	1
x	0	1	1
	1	0	1

$$x'y' + x'y = x'(y' + y) = x'(1) = x'$$

$$x'y + xy = (x' + x)y = (1)y = y$$

		y	
		0	1
x	0	1	1
	1	0	1

Subcube

- **Subcube** is a rectangle of adjacent 1-minterms:
 - Must be rectangular in shape
 - Number of 1-minterms enclosed must be a power of two

Diagram 1: A 2x4 Karnaugh map for variables x, y, z . The map shows four 1-minterms at $(x=0, yz=00), (x=0, yz=01), (x=0, yz=11), (x=0, yz=10)$. These four cells are enclosed by a dashed blue rectangle, representing a subcube where $x=0$. An arrow points to the top-right cell with the label x' .

F	yz	00	01	11	10
x	0	1	1	1	1
	1				

Diagram 2: A 2x4 Karnaugh map for variables x, y, z . The map shows four 1-minterms at $(x=0, yz=01), (x=0, yz=11), (x=1, yz=01), (x=1, yz=11)$. These four cells are enclosed by a dashed blue rectangle, representing a subcube where $z=1$. An arrow points to the top-right cell with the label z .

F	yz	00	01	11	10
x	0		1	1	
	1		1	1	

Diagram 3: A 2x4 Karnaugh map for variables x, y, z . The map shows four 1-minterms at $(x=0, yz=00), (x=0, yz=01), (x=1, yz=01), (x=1, yz=11)$. These four cells are enclosed by a dashed blue rectangle, representing a subcube where $x'y'=1$. Arrows point to the top-right cell with the label $x'y'$ and the bottom-right cell with the label $x'z$.

F	yz	00	01	11	10
x	0	1	1	1	
	1		1		

Diagram 4: A 2x4 Karnaugh map for variables x, y, z . The map shows four 1-minterms at $(x=0, yz=00), (x=0, yz=10), (x=1, yz=00), (x=1, yz=10)$. These four cells are enclosed by a dashed blue rectangle, representing a subcube where $z'=1$. An arrow points to the top-right cell with the label z' .

F	yz	00	01	11	10
x	0	1			1
	1	1			1

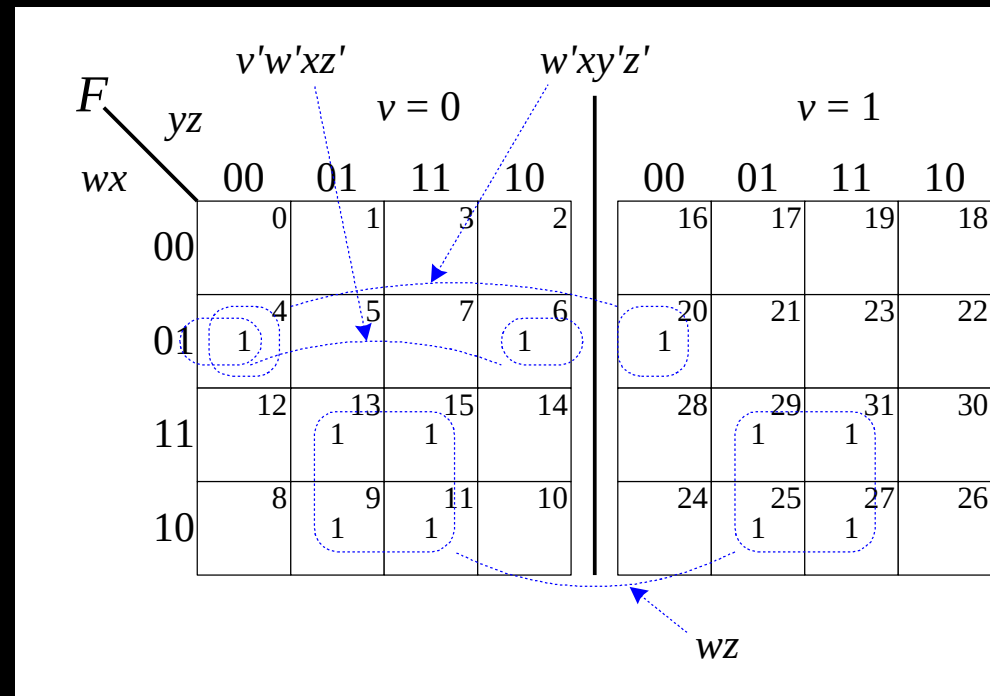
Diagram 5: A 4x4 Karnaugh map for variables w, x, y, z . The map shows four 1-minterms at $(wx=00, yz=00), (wx=00, yz=10), (wx=10, yz=00), (wx=10, yz=10)$. These four cells are enclosed by a dashed blue rectangle, representing a subcube where $x'z'=1$. An arrow points to the top-right cell with the label $x'z'$.

F	yz	00	01	11	10
wx	00	1			1
	01				
	11				
	10	1			1

Diagram 6: A 4x4 Karnaugh map for variables w, x, y, z . The map shows four 1-minterms at $(wx=00, yz=01), (wx=01, yz=01), (wx=11, yz=01), (wx=10, yz=01)$. These four cells are enclosed by a dashed blue rectangle, representing a subcube where $y'z=1$. An arrow points to the top-right cell with the label $y'z$.

F	yz	00	01	11	10
wx	00		1		
	01		1		
	11		1		
	10		1		

Subcube

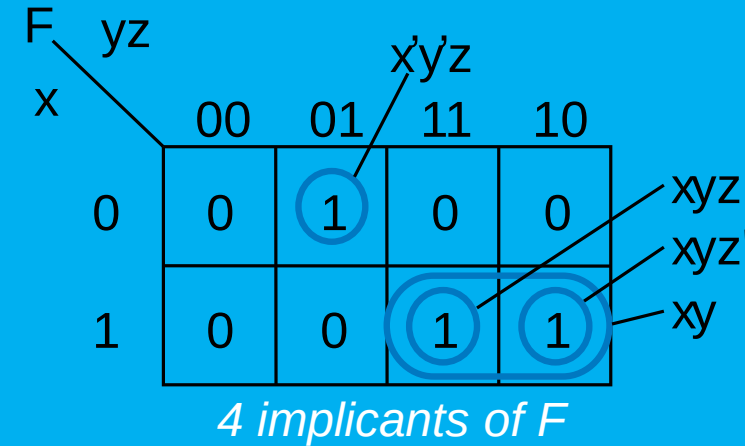


- Find as few subcubes as possible to cover all of the 1-minterms
- The size of the subcube should be as big as possible

Basic Concepts Underlying Automated Two-Level Logic Size Optimization

■ Definitions

- **On-set:** All minterms that define when $F=1$
- **Off-set:** All minterms that define when $F=0$
- **Implicant:** Any product term (minterm or other) that when 1 causes $F=1$
 - On K-map, any legal (but not necessarily largest) circle
 - Cover: Implicant xy **covers** minterms xyz and xyz'
- **Expanding** a term: removing a variable (like larger K-map circle)
 - $xyz \rightarrow xy$ is an expansion of xyz



Note: We use K-maps here just for intuitive illustration of concepts; automated tools do not use K-maps.

- **Prime implicant:** Maximally expanded implicant – any expansion would cover 1s not in on-set
 - $x'y'z$, and xy , above
 - But not xyz or xyz' – they can be expanded

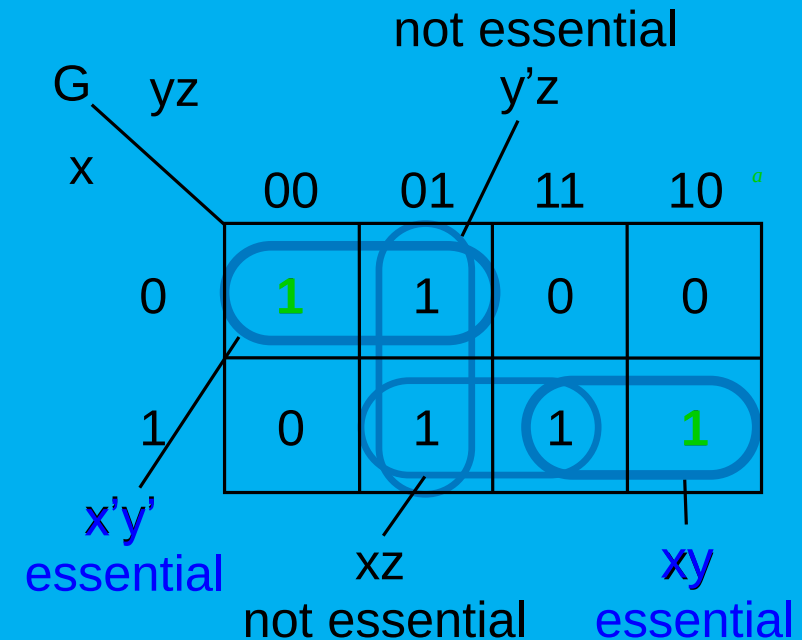
Basic Concepts Underlying Automated Two-Level Logic Size Optimization

■ Definitions (cont)

■ **Essential prime implicant:**

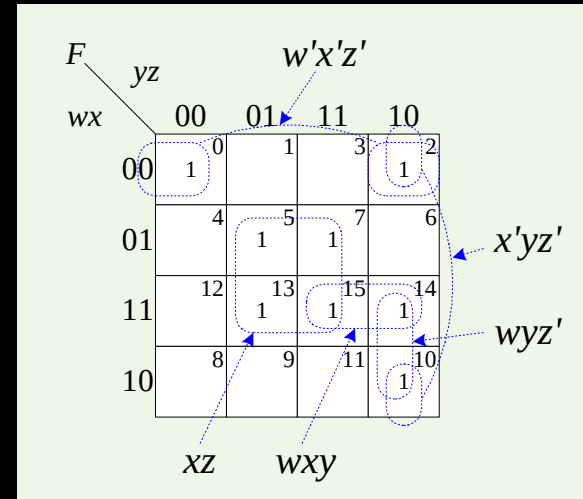
The only prime implicant that covers a **particular minterm** in a function's on-set

- Importance: We **must** include **all** essential PIs in a function's cover
- In contrast, some, but not all, non-essential PIs will be included



Procedure of using K-map

- Draw k-map
- Find prime implicant
- Find essential PI
- Create minimum cover
- Get the list



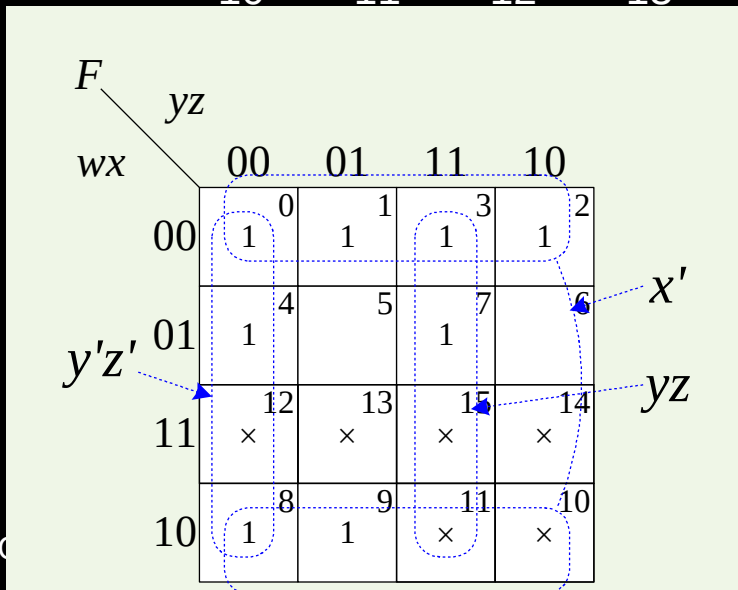
$$F = w'x'z' + xz + wyz'$$

$$F = z' (w'x' + wy) + xz$$

1-minterm	Prime Implicant	Essential PI
m_0	$w'x'z'$	$w'x'z'$
m_2	$w'x'z', x'yz'$	
m_5	xz	xz
m_7	xz	xz
m_{10}	$x'yz', wyz'$	
m_{13}	xz	xz
m_{14}	wyz', wxy	
m_{15}	xz	

Don't Care items

- **don't-care:** minterms we do not care whether their values are a 0 or a 1
- A function can be reduced even further using don't-cares
- Example:
 - 1-minterms: $m_0, m_1, m_2, m_3, m_4, m_7, m_8$, and m_9
 - $\times$ -minterms: $m_{10}, m_{11}, m_{12}, m_{13}, m_{14}$, and m_{15}



$$F = x' + yz + y'z'$$

$$F = x' + (y \odot z)$$

Don't Care Input Combinations

- What if we know that particular input combinations can never occur?
 - e.g., Minimize $F = xy'z'$, given that $x'y'z'$ ($xyz=000$) can *never* be true, and that $xy'z$ ($xyz=101$) can *never* be true
 - So it doesn't matter what F outputs when $x'y'z'$ or $xy'z$ is true, because those cases *will never occur*
 - Thus, make F be 1 or 0 for those cases *in a way that best minimizes the equation*

On K-map

- Draw **X**s for don't care combinations
 - Include X in circle ONLY if minimizes equation
 - Don't include other Xs

		yz		y'z'			
		00	01	11	10		
x	0	X	0	0	0		
	1	1	X	0	0		

Good use of don't cares

		yz		y'z'			
		00	01	11	10		
x	0	X	0	0	0		
	1	1	X	0	0		

unneeded

xy'

Unnecessary use of don't cares; results in extra term

Optimization Example using Don't Cares

- Minimize:
 - $F = a'bc' + abc' + a'b'c$
 - Given don't cares: **$a'bc$** , **abc**
- Note: Introduce don't cares with caution
 - Must be *sure* that we really don't care what the function outputs for that input combination
 - If we do care, even the slightest, then it's probably safer to set the output to

0

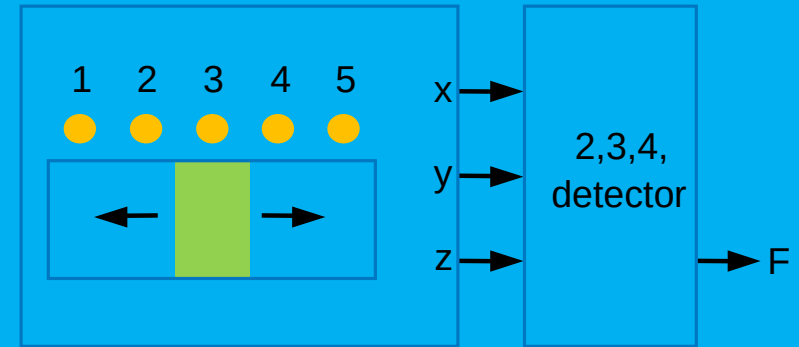
A Karnaugh map for a 3-variable function F(a, b, c). The vertical axis is labeled 'a' with values 0 and 1. The horizontal axis is labeled 'bc' with values 00, 01, 11, and 10. The map contains 1s at (a=0, bc=01), (a=0, bc=10), (a=1, bc=10), and (a=1, bc=11). The cells at (a=0, bc=11) and (a=1, bc=11) are marked with red 'X's, representing don't cares. Two blue circles are drawn: one around the 1s at (0,01) and (0,10), and another around the 1s at (1,10) and (1,11). Labels 'a'c' and 'b' point to the top-right and top-left corners of the map respectively.

	bc	00	01	11	10
a	0	0	1	X	1
1	0	0	0	X	1

$$F = a'c + b$$

Optimization with Don't Cares Example: Sliding Switch

- Switch with 5 positions
 - 3-bit value gives position in binary
- Want circuit that
 - Outputs 1 when switch is in position 2, 3, or 4
 - Outputs 0 when switch is in position 1 or 5
 - Note that the 3-bit input can never output binary 0, 6, or 7
 - Treat as don't care input combinations



Without don't cares:
 $F = x'y + xy'z'$

		yz			
	x	00	01	11	10
0		0	0	1	1
1		1	0	0	0

Labels: $x'y$ (points to cell 01, 11), $xy'z'$ (points to cell 11, 10)

With don't cares:
 $F = y + z'$

		yz			
	x	00	01	11	10
0		X	0	1	1
1		1	0	X	X

Labels: y (points to column 11), z' (points to column 10)

Quiz 3.1

Quine-McCluskey Method

Tabulation Method

Tabulation Method

- K-map is only good for at most maybe 5 variables
- Also known as Quine-McCluskey

$$F(w, x, y, z) = \Sigma(0, 2, 5, 7, 10, 13, 14, 15)$$

Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G ₀	m ₀	0	0	0	0	✓
G ₁	m ₂	0	0	1	0	✓
G ₂	m ₅	0	1	0	1	✓
	m ₁₀	1	0	1	0	✓
G ₃	m ₇	0	1	1	1	✓
	m ₁₃	1	1	0	1	✓
	m ₁₄	1	1	1	0	✓
G ₄	m ₁₅	1	1	1	1	✓



Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G ₀	m ₀ ,m ₂	0	0	–	0	
G ₁	m ₂ ,m ₁₀	–	0	1	0	
G ₂	m ₅ ,m ₇	0	1	–	1	✓
	m ₅ ,m ₁₃	–	1	0	1	✓
	m ₁₀ ,m ₁₄	1	–	1	0	
G ₃	m ₇ ,m ₁₅	–	1	1	1	✓
	m ₁₃ ,m ₁₅	1	1	–	1	✓
	m ₁₄ ,m ₁₅	1	1	1	–	

Tabulation Method

Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G_0	m_0, m_2	0	0	–	0	
G_1	m_2, m_{10}	–	0	1	0	
G_2	m_5, m_7	0	1	–	1	✓
	m_5, m_{13}	–	1	0	1	✓
	m_{10}, m_{14}	1	–	1	0	
G_3	m_7, m_{15}	–	1	1	1	✓
	m_{13}, m_{15}	1	1	–	1	✓
	m_{14}, m_{15}	1	1	1	–	

Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G_2	m_5, m_7, m_{13}, m_{15}	–	1	–	1	

- Five prime implicants (xz , $w'x'z'$, $x'yz'$, wyz' , and wxy) are exactly the same as those obtained in the Example
- Find the essential prime implicants and minimal list

Tabulation Method

Use tabulation method to simplify this equation:

$$F(w, x, y, z) = \Sigma(1, 2, 4, 5, 7, 13, 14)$$

Tabulation Method

Use tabulation method to simplify this equation:

$$F(w, x, y, z) = \Sigma(1, 2, 4, 5, 7, 13, 14)$$

Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G ₁	m ₁	0	0	0	1	✓
	m ₂	0	0	1	0	
	m ₄	0	1	0	0	✓
G ₂	m ₅	0	1	0	1	✓
G ₃	m ₇	0	1	1	1	✓
	m ₁₃	1	1	0	1	✓
	m ₁₄	1	1	1	0	

Tabulation Method

Use tabulation method to simplify this equation:

$$F(w, x, y, z) = \Sigma(1, 2, 4, 5, 7, 13, 14)$$

Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G ₁	m ₁	0	0	0	1	✓
	m ₂	0	0	1	0	
	m ₄	0	1	0	0	✓
G ₂	m ₅	0	1	0	1	✓
G ₃	m ₇	0	1	1	1	✓
	m ₁₃	1	1	0	1	✓
	m ₁₄	1	1	1	0	



Group	Subcube Minterms	Subcube Value				Subcube Covered
		w	x	y	z	
G ₁	m ₁ , m ₅	0	–	0	1	
	m ₄ , m ₅	0	1	0	–	
G ₂	m ₅ , m ₇	0	1	–	1	
	m ₅ , m ₁₃	–	1	0	1	

K-Map

Use the K-map method to simplify this equation:

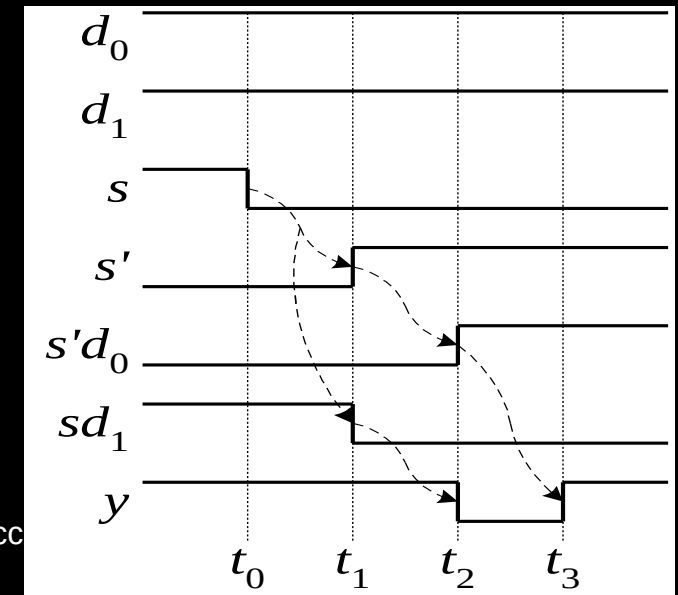
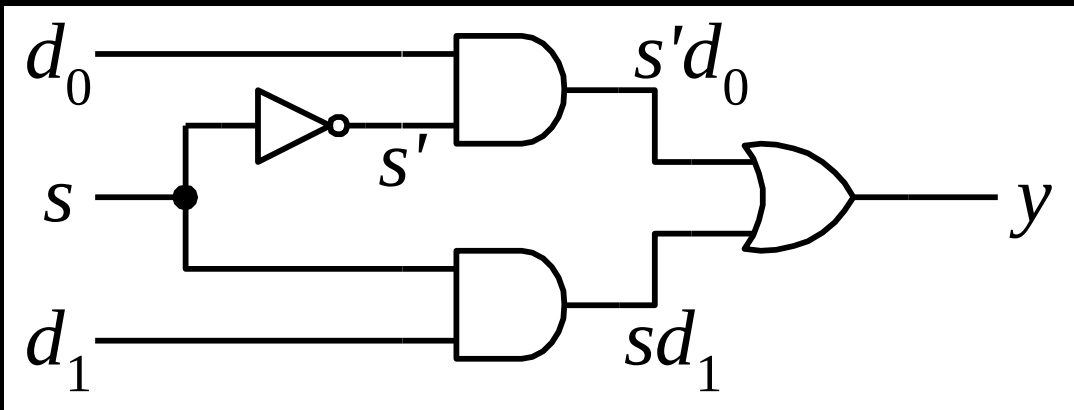
$$F(w, x, y, z) = \Sigma(1, 2, 4, 5, 7, 13, 14)$$

F		yz			
		wx	00	01	11
wx	00		1		1
	01	1	1	1	
	11		1		1
	10				

Quiz 3.2

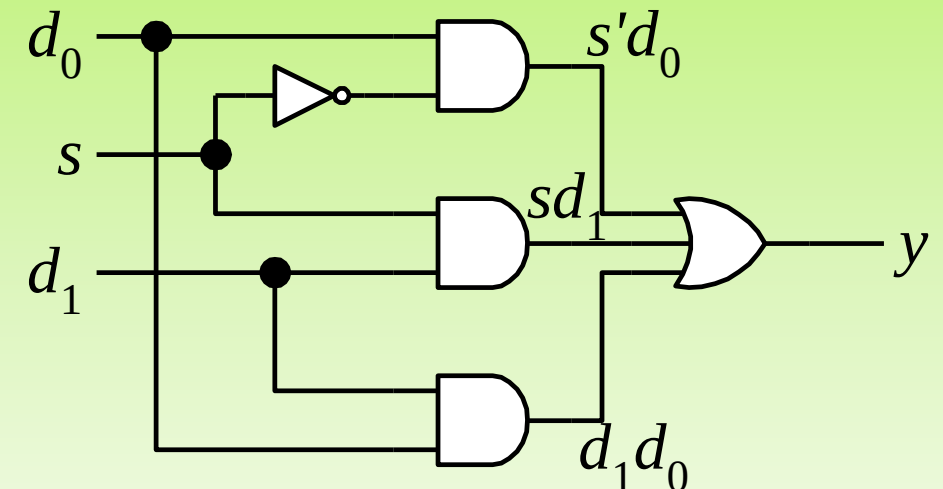
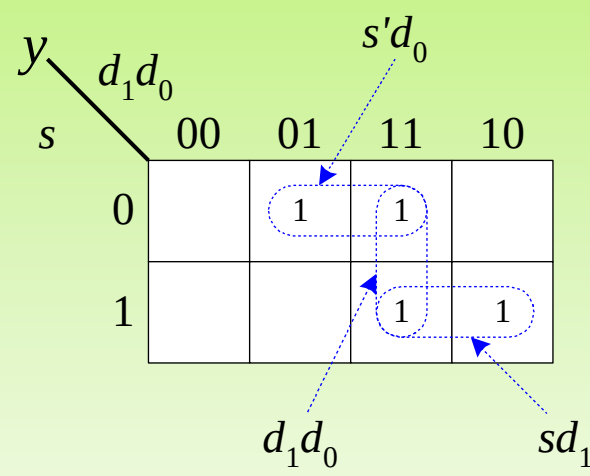
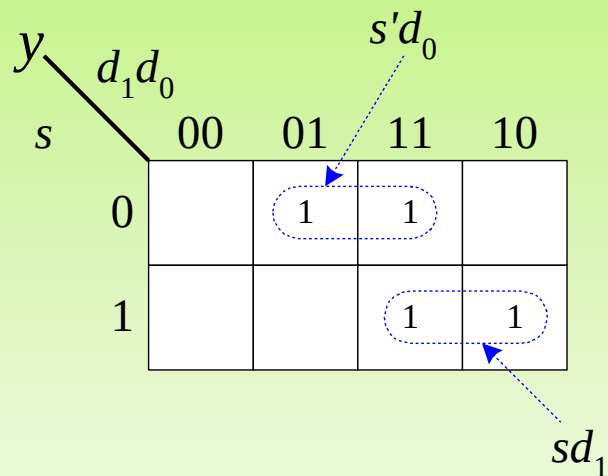
Time Hazards and Glitches

- Functional analysis VS Timing analysis (Ideal circuit VS actual implementation)
- **Timing hazards** are problems in a circuit as a result of timing issues.
- A **glitch** is when a signal is expected to be stable (from a functional analysis), but it changes value for a brief moment and then goes back to what it is expected to be.



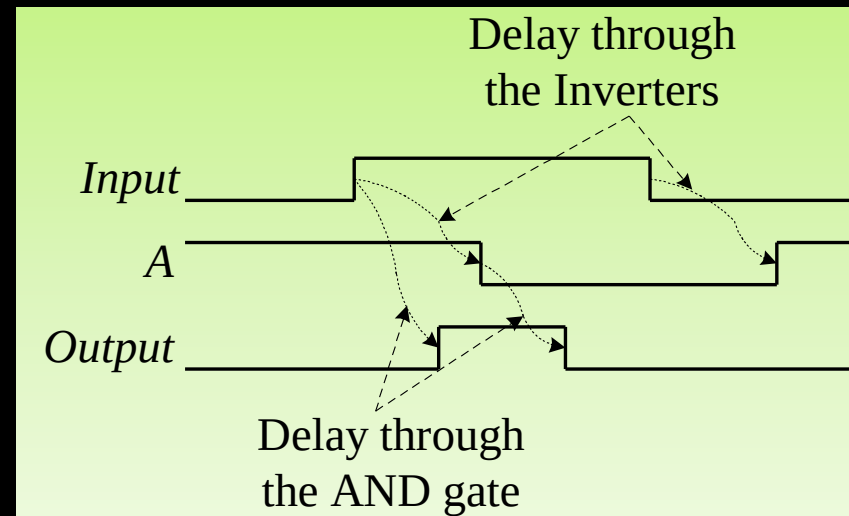
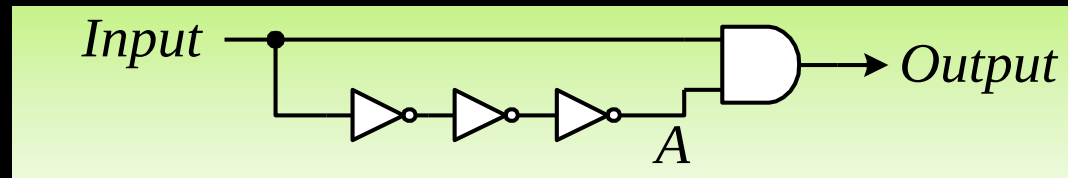
Time Hazards and Glitches

- Glitches are caused by multiple sources having paths of different delays
- A glitch can be eliminated by adding an extra prime implicant
- Equivalent to adding an extra path between the source and the destination



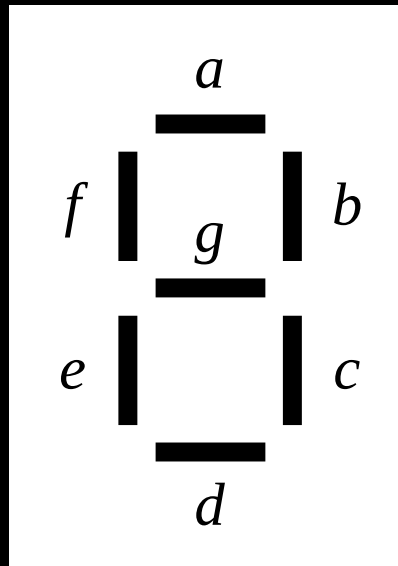
Using Glitch for One-shot

- A circuit that outputs a single, short pulse when given an input of arbitrary time length is known as a *one-shot*.



BCD to 7-Segment Decoder

- Converts a 4-bit binary coded decimal (BCD) input to seven output signals for turning on the seven LEDs in a 7-segment LED display
- A typical combinational circuit synthesis problem



BCD to 7-Segment Decoder

Inputs				Decimal Digit	Display
i_3	i_2	i_1	i_0		
0	0	0	0	0	
0	0	0	1	1	
0	0	1	0	2	
0	0	1	1	3	
0	1	0	0	4	
0	1	0	1	5	
0	1	1	0	6	
0	1	1	1	7	
1	0	0	0	8	
1	0	0	1	9	
Rest of the Combinations					

BCD to 7-Segment Decoder

Inputs				Decimal Digit	Display	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
i_3	i_2	i_1	i_0									
0	0	0	0	0								
0	0	0	1	1								
0	0	1	0	2								
0	0	1	1	3								
0	1	0	0	4								
0	1	0	1	5								
0	1	1	0	6								
0	1	1	1	7								
1	0	0	0	8								
1	0	0	1	9								
Rest of the Combinations												

HDL for BCD to 7-Segment Decoder

- Transistor
- Gate
- RTL
- Behavior
- Structural
- Dataflow
- Behavior

HDL for BCD to 7-Segment Decoder

- **Structural**
- Dataflow
- Behavior

- The lowest level, manually design the circuit
- Specify the components and gates and how they are connected
- Produce a netlist that is like your original circuit
- Advantage: you have full control
- Disadvantage: manually come up with the circuit

HDL for BCD to 7-Segment Decoder

- Structural
- Dataflow
- Behavior

```
module bcd (  
    input i0, i1, i2, i3,  
    output a, b, c, d, e, f, g  
);  
    wire ip0,ip1,ip2,a1,b1,d1,d2,d3,d4,e1,e2,f1,f2,f3,g1,g2;  
  
    // first parameter is the output; remaining parameters are the inputs  
    not U0(ip0,i0);  
    not U1(ip1,i1);  
    not U2(ip2,i2);  
    xnor U3(a1,i2,i0);  
    or U4(a,i3,i1,a1);  
    xnor U5(b1,i1,i0);  
    or U6(b,ip2,b1);  
    or U7(c,i2,ip1,i0);  
    and U8(d1,i1,ip0);  
    and U9(d2,ip2,ip0);  
    ...  
endmodule
```

HDL for BCD to 7-Segment Decoder

- Uses concurrent assign statements to assign values to the output signals based on the Boolean equations
- Structural
- Dataflow
- Behavior

$$a = i_3 + i_1 + (i_2 \odot i_0) \quad \text{assign } a = i3 \mid i1 \mid (i2 \sim^{\wedge} i0);$$

```
module bcd (  
    input i0, i1, i2, i3,  
    output a, b, c, d, e, f, g  
);  
    assign a = i3 | i1 | (i2 ~^ i0);           // bitwise ~=NOT; &=AND; |=OR; ^=XOR  
    assign b = ~i2 | ~(i1 ^ i0);  
    assign c = i2 | ~i1 | i0;  
    assign d = (i1 & ~i0) | (~i2 & ~i0) | (~i2 & i1) | (i2 & ~i1 & i0);  
    assign e = (i1 & ~i0) | (~i2 & ~i0);  
    assign f = i3 | (i2 & ~i1) | (i2 & ~i0) | (~i1 & ~i0) | (i2 & ~i0) | (~i1 & ~i0);  
    assign g = i3 | (i2 ^ i1) | (i1 & ~i0);  
endmodule
```


HDL for BCD to 7-Segment Decoder

- Structural
- Dataflow
- Behavior

```
module bcd (  
    input [3:0] I,  
    output reg [0:6] Segs );  
    always @ (I) begin  
        case (I)  
            0: Segs = 7'b1111110;  
            1: Segs = 7'b0110000;  
            2: Segs = 7'b1101101;  
            3: Segs = 7'b1111001;  
            4: Segs = 7'b0110011;  
            5: Segs = 7'b1011011;  
            6: Segs = 7'b1011111;  
            7: Segs = 7'b1110000;  
            8: Segs = 7'b1111111;  
            9: Segs = 7'b1110011;  
            default: Segs = 7'b0000000;  
        endcase  
    end  
endmodule
```

Conclusions

- Analysis of Combinational Circuits
- Synthesis of Combinational Circuits
- Minimization
- Time Hazards and Glitches
- Example: BCD to 7-segment Decoder
- Verilog Code for example